

LAPORAN PRATIKUM PEKAN 8

STRUKTUR DATA

disusun Oleh:

ALIF MUHAMMAD IHSAN

NIM 2511531004

Dosen Pengampu : Dr.Wahyudi S.T, M.T

Asisten : M Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2025

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum **Struktur Data** ini dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban atas kegiatan praktikum yang telah dilaksanakan, sekaligus sebagai sarana untuk memperdalam pemahaman mengenai konsep dasar struktur data, implementasi algoritma, serta penerapan logika pemrograman dalam kehidupan nyata.

Dalam penyusunan laporan ini, penulis menyadari masih terdapat banyak kekurangan, baik dari segi isi maupun penyajian. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang.

Ucapan terima kasih penulis sampaikan kepada:

- Dosen pengampu mata kuliah **Struktur Data** yang telah memberikan arahan dan bimbingan.
- Asisten praktikum yang senantiasa membantu dalam memahami materi dan menyelesaikan permasalahan teknis.
- Rekan-rekan mahasiswa yang turut berdiskusi dan bekerja sama selama kegiatan praktikum berlangsung.

Akhir kata, semoga laporan ini dapat memberikan manfaat, baik bagi penulis sendiri maupun bagi pembaca yang ingin mempelajari lebih lanjut mengenai struktur data.

Daftar Isi

KATA PENGANTAR.....	i
BAB I.....	3
PENDAHULUAN	3
1.1 Latar Belakang.....	3
1.2 Tujuan.....	4
1.3 Manfaat Praktikum	4
BAB II.....	5
PEMBAHASAN.....	5
2.1 Kode Program	5

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu aspek fundamental dalam ilmu komputer yang berperan penting dalam pengelolaan dan pengorganisasian data. Dengan adanya struktur data, proses penyimpanan, pengambilan, serta manipulasi data dapat dilakukan secara lebih efisien dan terstruktur. Pemahaman mengenai struktur data menjadi dasar bagi pengembangan algoritma yang optimal, sehingga sangat berpengaruh terhadap performa suatu program maupun sistem.

Melalui praktikum struktur data, mahasiswa tidak hanya mempelajari konsep teoritis, tetapi juga mengimplementasikan langsung berbagai bentuk struktur data seperti array, linked list, stack, queue, dan tree dalam bahasa pemrograman. Praktikum ini bertujuan untuk melatih keterampilan analisis logika, pemecahan masalah, serta kemampuan debugging yang diperlukan dalam pengembangan perangkat lunak.

Selain itu, praktikum ini juga memberikan pengalaman nyata dalam menghubungkan teori dengan praktik, sehingga mahasiswa dapat memahami bagaimana suatu algoritma bekerja di balik layar dan bagaimana pemilihan struktur data yang tepat dapat memengaruhi efisiensi program. Dengan demikian, kegiatan praktikum struktur data menjadi sarana penting untuk membangun fondasi keilmuan yang kuat dalam bidang informatika.

1.2 Tujuan

1. Memahami konsep dasar struktur data dan penerapannya dalam pemrograman.
2. Melatih keterampilan dalam mengimplementasikan berbagai jenis struktur data seperti array, linked list, stack, queue, dan tree menggunakan bahasa pemrograman.
3. Mengembangkan kemampuan analisis logika dan pemecahan masalah melalui studi kasus yang diberikan dalam praktikum.
4. Melatih keterampilan debugging serta troubleshooting program agar dapat menemukan dan memperbaiki kesalahan secara sistematis.

1.3 Manfaat Praktikum

1. Memberikan pengalaman langsung dalam mengimplementasikan struktur data sehingga mahasiswa lebih mudah dalam memahami konsep yang abstrak
2. Meningkatkan keterampilan pemograman, khususnya dalam efesiensi algoritma dan pengelolaan data.
3. Menjadi bekal penting untuk mata kuliah lanjutan maupun proyek pengembangan perangkat lunak di masa depan.
4. Membantu mahasiswa memahami pentingnya pemilihan struktur data yang tepat dalam meningkatkan peforma program.

BAB II

PEMBAHASAN

2.1 Kode Program

a. mergeSort_2511531004

```
package Pekan8_2511531004;

public class mergeSort_2511531004 {

    void merge_2511531004 (int[] arr, int l , int m , int r) {
        int n1_1004 = m - l + 1;
        int n2_1004 = r - m;

        int L_1004[] = new int[n1_1004];
        int R_1004[] = new int[n2_1004];

        for (int i = 0; i < n1_1004; ++i)
            L_1004[i] = arr[l + i];
        for (int j = 0; j < n2_1004; ++j)
            R_1004[j] = arr[m + 1 + j];

        int i = 0, j = 0;
        int k = l;

        while (i < n1_1004 && j < n2_1004) {
            if (L_1004[i] <= R_1004[j]) {
                arr[k] = L_1004[i];
                i++;
            } else {
                arr[k] = R_1004[j];
                j++;
            }
            k++;
        }

        while (i < n1_1004) {
            arr[k] = L_1004[i];
            i++;
            k++;
        }

        while (j < n2_1004) {
            arr[k] = R_1004[j];
            j++;
            k++;
        }
    }

    void sort_2511531004 (int[] arr, int l , int r) {
        if (l < r) { // ☒ diperbaiki
            int m = l + (r - l) / 2; // ☒ diperbaiki
```

```

        sort_2511531004(arr, l, m);
        sort_2511531004(arr, m + 1, r);
        merge_2511531004(arr, l, m, r);
    }
}

static void printArray_2511531004 (int arr[]) {
    int n = arr.length;
    for(int i = 0; i < n ; ++i)
        System.out.print(arr[i] + " ");
    System.out.println();
}

public static void main(String[] args) {
    int arr[] = {12, 11, 13, 5, 6, 7};
    System.out.println("Data sebelum diurutkan:");
    printArray_2511531004(arr);

    mergeSort_2511531004 ob = new mergeSort_2511531004();
    ob.sort_2511531004(arr, 0, arr.length - 1);

    System.out.println("Data setelah diurutkan:");
    printArray_2511531004(arr);
}
}

```

b. quicksort_2511531004

```

package Pekan8_2511531004;

public class QuickShort_2511531004 {

    public static void swap_2511531004 (int [] arr, int i , int j ) {
        int temp_1004 = arr[i] ;
        arr[i] = arr[j];
        arr[j] = temp_1004;
    }

    // metode tambahan untuk mengatur pivot menggunakan median median
of three
    static void medianOfThree_2511531004 (int [] arr, int low, int
high) {
        int mid_1004 = low + (high - low) / 2;

        // urutkan elemen low, mid, dan high
        if (arr[low] > arr[mid_1004]) {
            swap_2511531004(arr, low, mid_1004);
        }
        if (arr[low] > arr[high]) {
            swap_2511531004(arr, low, high);
        }
        if (arr[mid_1004] > arr[high]) {
            swap_2511531004(arr, mid_1004, high);
        }
        swap_2511531004(arr, mid_1004, high);
    }
}

```

```

    }

    static int partition_2511531004 (int[] arr, int low, int high) {
        // panggil fungsi median of three sebelum menentukan pivot
        medianOfThree_2511531004(arr, low, high);

        int pivot = arr[high]; // sekarang arr[high] sudah berisi
        nilai median

        int i = (low - 1);

        for (int j = low; j <= high - 1 ; j++) {
            // jika elemen saat ini lebih kecil dari atau sama
            dengan pivot

            if (arr[j] < pivot) {
                // increment indeks elemen yang lebih kecil
                i++;
                swap_2511531004(arr, i , j);
            }
        }
        swap_2511531004(arr, i + 1, high);
        return (i + 1);
    }

    public static void quickSort_2511531004 (int[] arr, int low , int
    high) {
        if (low < high) {
            int pi = partition_2511531004(arr, low, high);
            quickSort_2511531004(arr, low, pi - 1);
            quickSort_2511531004(arr, pi + 1, high) ;
        }
    }

    public static void printArr_2511531004(int[] arr) {
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        int[] arr = {10,7,8,9,1,5} ;
        int N = arr.length;
        System.out.print("Data sebelum diurutkan : ");
        printArr_2511531004(arr);

        quickSort_2511531004 (arr, 0, N - 1);

        System.out.print("data setelah diurutkan : ");
        printArr_2511531004(arr);

    }
}

```


c. shellSort_2511531004

```
package Pekan8_2511531004;

public class shellSort_2511531004 {

    public static void shellSort_2511531004_( int[] A) {
        int n_1004 = A.length;
        int gap_1004 = n_1004 / 2;

        while ( gap_1004 > 0 ) {
            for (int i = gap_1004; i < n_1004 ; i++) {
                int temp = A[i];
                int j = i;
                while (j >= gap_1004 && A[j - gap_1004] >
temp) {
                    A[j] = A[j-gap_1004];
                    j = j - gap_1004;
                }
                A[j] = temp;
            }
            gap_1004 = gap_1004/2;
        }

    }

    public static void main(String [] args) {
        int [] data_1004 = {3,10,4,6,0,9,7,2,1,5} ;
        System.out.print("sebelum : ");
        printArray_2511531004(data_1004);

        shellSort_2511531004(data_1004);

        System.out.print("Sesudah (Shell Sort) : ");
        printArray_2511531004(data_1004);

    }

    public static void printArray_2511531004 (int[] arr) {
        for (int i : arr) System.out.print(i + " ");
        System.out.println();
    }
}
```

d. BubbleSortGUI_2511531004

```
package Pekan8_2511531004;
```

```
import javax.swing.*;
```

```
import java.awt.*;
```

```
import java.util.ArrayList;
```

```
public class BubleSortGUI_2511531004 extends JFrame {
```

```
    private static final long serialVersionUID = 1L;
```

```
    private int [] array_1004;
```

```
    private JLabel[] labelArray_1004;
```

```
    private JButton stepButton_1004, resetButton_1004, setButton_1004;
```

```
    private JTextField inputField_1004;
```

```
    private JPanel panelArray_1004;
```

```
    private JTextArea stepArea_1004;
```

```
    private JPanel contentPane;
```

```
    private int i_1004 = 1,j_1004;
```

```
    private boolean sorting_1004 = false;
```

```
    private int stepCount_1004;
```

```
    /**
```

```

* Launch the application.

*/

/**

* Create the frame.

*/

public BubleSortGUI_2511531004() {

    setTitle("Insertion Sort Langkah per Langkah");

    setSize(750, 400);

    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    setLocationRelativeTo(null);

    setLayout(new BorderLayout());

    JPanel inputPanel_1004 = new JPanel(new FlowLayout());

    inputField_1004 = new JTextField(30);

    setButton_1004 = new JButton("Set Array");

    inputPanel_1004.add(new JLabel("Masukkan angka (pisahkan dengan
koma):"));

    inputPanel_1004.add(inputField_1004);

    inputPanel_1004.add(setButton_1004);

    panelArray_1004 = new JPanel();

    panelArray_1004.setLayout(new FlowLayout());

```

```

JPanel controlPanel_1004 = new JPanel();

stepButton_1004 = new JButton("Langkah Selanjutnya");

resetButton_1004 = new JButton("Reset");

stepButton_1004.setEnabled(false);

controlPanel_1004.add(stepButton_1004);

controlPanel_1004.add(resetButton_1004);


stepArea_1004 = new JTextArea(8, 60);

stepArea_1004.setEditable(false);

stepArea_1004.setFont(new Font("Monospaced", Font.PLAIN, 14));

JScrollPane scrollPane_1004 = new JScrollPane(stepArea_1004);


add(inputPanel_1004, BorderLayout.NORTH);

add(panelArray_1004, BorderLayout.CENTER);

add(controlPanel_1004, BorderLayout.SOUTH);

add(scrollPane_1004, BorderLayout.EAST);


// Event Set Array

setButton_1004.addActionListener(e_1004                                ->
setArrayFromInput_1004());

// Event Langkah Selanjutnya

stepButton_1004.addActionListener(e_1004 -> performStep_1004());

```

```

// Event Reset

resetButton_1004.addActionListener(e_1004 -> reset_1004());
}

private void performStep_1004() {
    if (!sorting_1004 || j_1004 >= array_1004.length - 1) {
        sorting_1004 = false;

        stepButton_1004.setEnabled(false);

        JOptionPane.showMessageDialog(this, "Sorting selesai!");

        return;
    }

    resetHighlights_1004();

    StringBuilder stepLog = new StringBuilder();

    labelArray_1004[j_1004].setBackground(Color.CYAN);
    labelArray_1004[j_1004 + 1].setBackground(Color.CYAN);

    if (array_1004[j_1004] > array_1004[j_1004 + 1]) {
        // Swap

        int temp = array_1004[j_1004];
        array_1004[j_1004] = array_1004[j_1004 + 1];
        array_1004[j_1004 + 1] = temp;

        labelArray_1004[j_1004].setBackground(Color.RED);
        labelArray_1004[j_1004 + 1].setBackground(Color.RED);

        stepLog.append("Langkah   ").append(stepCount_1004).append(":
Menukar elemen ke-")

```

```

        .append(j_1004).append(" ").append(array_1004[j_1004 +
1]).append(") dengan ke-")

        .append(j_1004 + 1).append("
").append(array_1004[j_1004]).append(")");

    } else {

        stepLog.append("Langkah ").append(stepCount_1004).append(":
Tidak ada pertukaran antara ke-")

        .append(j_1004).append(" dan ke-").append(j_1004 +
1).append("\n");

    }

    stepLog.append("Hasil:
").append(arrayToString_1004(array_1004)).append("\n\n");

    stepArea_1004.append(stepLog.toString());

    updateLabels_1004();

    j_1004++;

    if (j_1004 >= array_1004.length - i_1004 - 1) {

        j_1004 = 0;

        i_1004++;

    }

    stepCount_1004++;

    if (i_1004 >= array_1004.length - 1) {

        sorting_1004 = false;

        stepButton_1004.setEnabled(false);

        JOptionPane.showMessageDialog(this, "Sorting selesai!");

    }

```

```

    }

    private void updateLabels_1004() {
        for (int k = 0; k < array_1004.length; k++) {

labelArray_1004[k].setText(String.valueOf(array_1004[k]));

        }
    }

    private void resetHighlights_1004() {
        for (JLabel label: labelArray_1004) {
            label.setBackground(Color.white);
        }
    }

    private void reset_1004 () {
        inputField_1004.setText("");
        panelArray_1004.removeAll();
        panelArray_1004.revalidate();
        panelArray_1004.repaint();
        stepArea_1004.setText("");
        stepButton_1004.setEnabled(false);
        sorting_1004 = false;
        i_1004 = 0;
    }

```

```

        j_1004 = 0;

        stepCount_1004 = 1;

    }

private String arrayToString_1004(int[] arr_1004) {

    StringBuilder sb = new StringBuilder();

    for (int k = 0; k < arr_1004.length; k++) {

        sb.append(arr_1004[k]);

        if (k < arr_1004.length - 1) sb.append(", ");

    }

    return sb.toString();

}

private void setArrayFromInput_1004() {

    String text = inputField_1004.getText().trim();

    if (text.isEmpty()) return;

    String[] parts = text.split(",");

    array_1004 = new int[parts.length];

    try {

        for (int k = 0; k < parts.length; k++) {

            array_1004[k] = Integer.parseInt(parts[k].trim());

        }

    } catch (NumberFormatException e) {

        // Handle the exception
    }

}

```



```

    }

    } catch (NumberFormatException e) {

        JOptionPane.showMessageDialog(this, "Masukkan hanya angka "
            + "yang dipisahkan koma", "Error",
JOptionPane.ERROR_MESSAGE);

        return;

    }

    i_1004 = 0;

    j_1004 = 0;

    stepCount_1004 = 1;

    sorting_1004 = true;

    stepButton_1004.setEnabled(true);

    stepArea_1004.setText("");

    panelArray_1004.removeAll();

    labelArray_1004 = new JLabel[array_1004.length];

    for (int k = 0; k < array_1004.length; k++) {

        labelArray_1004[k] = new JLabel(String.valueOf(array_1004[k]));

        labelArray_1004[k].setFont(new Font("Arial", Font.BOLD, 24));

        labelArray_1004[k].setOpaque(true);

        labelArray_1004[k].setBackground(Color.WHITE);

labelArray_1004[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));

        labelArray_1004[k].setPreferredSize(new Dimension(50, 50));

```

```
labelArray_1004[k].setHorizontalAlignment(SwingConstants.CENTER);
```

```
    panelArray_1004.add(labelArray_1004[k]);
```

```
    }
```

```
    panelArray_1004.revalidate();
```

```
    panelArray_1004.repaint();
```

```
}
```

```
public static void main(String [] args) {
```

```
    SwingUtilities.invokeLater (() -> {
```

```
        BubleSortGUI_2511531004    gui_1004    =    new  
BubleSortGUI_2511531004() ;
```

```
        gui_1004.setVisible(true);
```

```
    });
```

```
}
```

}

2.2 Langkah Kerja

a. mergeSort_2511531004

1. Mulai dari array penuh

- Data awal: {12, 11, 13, 5, 6, 7}
- Fungsi `sort_2511531004(arr, 0, arr.length-1)` dipanggil.

2. Bagi array jadi dua bagian (Divide)

- Hitung $mid = 1 + (r-l)/2$.
- Array dibagi:
 - Kiri: {12, 11, 13}
 - Kanan: {5, 6, 7}

3. Rekursi ke bagian kiri

- {12, 11, 13} dibagi lagi:
 - Kiri: {12, 11}
 - Kanan: {13}
- {12, 11} dibagi:
 - {12} dan {11} → lalu digabung jadi {11, 12}.
- Gabungkan dengan {13} → hasil kiri jadi {11, 12, 13}.

4. Rekursi ke bagian kanan

- {5, 6, 7} dibagi:
 - Kiri: {5, 6}
 - Kanan: {7}
- {5, 6} dibagi:
 - {5} dan {6} → digabung jadi {5, 6}.

- Gabungkan dengan {7} → hasil kanan jadi {5, 6, 7}.

5. Gabungkan hasil kiri dan kanan (Conquer + Merge)

- Kiri: {11, 12, 13}
- Kanan: {5, 6, 7}
- Digabung dengan perbandingan elemen satu per satu:
 - Ambil 5 dulu (lebih kecil dari 11)
 - Lalu 6
 - Lalu 7
 - Baru sisanya 11, 12, 13
- Hasil akhir: {5, 6, 7, 11, 12, 13}

b. quicksort_2511531004

1. Data awal

2. {10, 7, 8, 9, 1, 5}

3. Pilih pivot dengan median-of-three

- Ambil tiga elemen: low, mid, high.
 - low = 10, mid = 8, high = 5
- Urutkan ketiganya → median = 8.
- Pivot dipasang di posisi high.

4. Partisi array

- Bandingkan tiap elemen dengan pivot (8).
- Elemen $\leq$ pivot dipindah ke kiri, $>$ pivot ke kanan.
- Setelah partisi pertama:
- {7, 5, 1, 8, 9, 10}

Pivot (8) sudah di posisi benar.

5. Rekursi ke kiri (low..pi-1)

- Subarray kiri: {7, 5, 1}
- Median-of-three $\rightarrow$ pivot = 5.
- Partisi $\rightarrow$ {1, 5, 7}

6. Rekursi ke kanan (pi+1..high)

- Subarray kanan: {9, 10}
- Median-of-three $\rightarrow$ pivot = 9.
- Partisi $\rightarrow$ {9, 10} (sudah terurut).

7. Gabungkan hasil

- Kiri: {1, 5, 7}
- Pivot utama: 8
- Kanan: {9, 10}
- Hasil akhir:
- {1, 5, 7, 8, 9, 10}

c. shellSort_2511531004

1. Data awal

2. {3, 10, 4, 6, 0, 9, 7, 2, 1, 5}

Panjang array = 10 $\rightarrow$ gap pertama = $10/2 = 5$.

2. Iterasi gap = 5

- Bandingkan elemen yang berjarak 5 posisi.
- Misalnya: A[0] dengan A[5], A[1] dengan A[6], dst.
- Setelah proses, elemen mulai bergeser mendekati posisi yang benar.

3. Iterasi **gap = 2**

- Sekarang bandingkan elemen yang berjarak 2.
- Proses ini lebih halus, memperbaiki urutan yang masih salah.

4. Iterasi **gap = 1 (Insertion Sort)**

- Gap terakhir = 1 → sama seperti **Insertion Sort**.
- Semua elemen dibandingkan dengan tetangganya dan disisipkan ke posisi yang tepat.
- Hasil akhir array terurut ascending.

d. BubleSortGUI_2511531004

1. **Input Array**

- User masukkan angka di `inputField_1004` (misalnya: 5,3,8,1).
- Tombol **Set Array** akan parsing input → simpan ke `array_1004` → tampilkan ke panel dengan `JLabel`.

2. **Mulai Sorting**

- Klik tombol **Langkah Selanjutnya** → program masuk ke `performStep_1004()`.
- Variabel kontrol:
 - `i_1004` → menandai pass ke berapa.
 - `j_1004` → menandai posisi elemen yang dibandingkan.
 - `stepCount_1004` → menghitung jumlah langkah.

3. **Proses Perbandingan**

- Dua elemen bersebelahan (`array[j]` dan `array[j+1]`) di-highlight warna **cyan**.
- Jika `array[j] > array[j+1]` → dilakukan **swap** (pertukaran).

- Label berubah warna **merah** untuk menandai ada pertukaran.
- Jika tidak ada pertukaran → log menampilkan "Tidak ada pertukaran".

4. Update Tampilan

- Setelah tiap langkah, updateLabels_1004() memperbarui angka di GUI.
- stepArea_1004 menampilkan log detail:
- Langkah 1: Menukar elemen ke-0 (5) dengan ke-1 (3)
- Hasil: 3, 5, 8, 1

5. Iterasi Pass

- j_1004 bergerak dari kiri ke kanan.
- Jika sudah sampai akhir ($j \geq \text{array.length} - i - 1$), maka j reset ke 0 dan i bertambah → masuk ke pass berikutnya.
- Proses diulang sampai i mencapai $\text{array.length} - 1$.

6. Selesai

- Jika semua pass selesai, tombol **Langkah Selanjutnya** otomatis disable.
- Muncul pesan JOptionPane: "**Sorting selesai!**".
- Array sudah terurut ascending.

BAB III

PENUTUP

3.1 Kesimpulan

- **Algoritma Sorting** seperti Merge Sort, Quick Sort, Shell Sort, dan Bubble Sort memiliki cara kerja berbeda, tetapi tujuan utamanya sama yaitu mengurutkan data agar lebih terstruktur dan mudah diproses.
- **Merge Sort** menggunakan pendekatan *divide and conquer*, membagi array menjadi bagian kecil lalu menggabungkannya kembali secara terurut.
- **Quick Sort** juga berbasis *divide and conquer*, tetapi lebih efisien dengan memilih pivot (median-of-three) untuk meminimalisir kasus terburuk.
- **Shell Sort** adalah pengembangan dari Insertion Sort dengan gap yang mengecil, sehingga mempercepat proses pengurutan.
- **Bubble Sort GUI** memberikan visualisasi langkah demi langkah, sehingga memudahkan pemahaman konsep pertukaran elemen dalam proses sorting.

3.2 Saran

- **Optimalkan penggunaan algoritma** sesuai kebutuhan:
 - Quick Sort untuk data besar dengan performa tinggi.
 - Merge Sort untuk data yang butuh stabilitas hasil.
 - Shell Sort untuk data sedang dengan efisiensi lebih baik dari Insertion Sort.
 - Bubble Sort hanya cocok untuk pembelajaran karena kurang efisien.
- **Tambahkan fitur pilihan algoritma** dalam satu program agar user bisa membandingkan hasil dan performa secara langsung.
- **Gunakan visualisasi GUI** (seperti Bubble Sort GUI) untuk algoritma lain juga, sehingga konsep *divide and conquer* atau *gap sequence* lebih mudah dipahami.
- **Uji dengan dataset lebih besar dan beragam** (misalnya 50–100 elemen) untuk melihat perbedaan waktu eksekusi antar algoritma.
- **Dokumentasikan kompleksitas waktu dan ruang** dari tiap algoritma di laporan, agar analisis lebih lengkap dan sesuai kriteria penilaian.

DAFTAR PUSTAKA

Oracle. (2024). *Java™ Platform, Standard Edition Documentation*. Oracle.
<https://docs.oracle.com/javase/8/docs/api/>

Deitel, P. J., & Deitel, H. M. (2017). *Java: How to Program (Early Objects)* (11th ed.). Pearson Education.

Liang, Y. D. (2019). *Introduction to Java Programming and Data Structures* (11th ed.). Pearson.

Wirth, N. (2006). *Algorithms and Data Structures*. Springer.

GeeksforGeeks. (2025). *Java Linked List – Insertion, Deletion, and Search Operations*. <https://www.geeksforgeeks.org/data-structures/linked-list/>

TutorialsPoint. (2025). *Java – Linked List Implementation*.

Tugas Struktur Data Pekan 5

a. Kode Program Lagu_2511531004

```
package Pekan8_2511531004;

public class Lagu_2511531004 {
    String judul_1004;
    String penyanyi_1004;
    int durasi_1004;

    Lagu_2511531004(String judul, String penyanyi, int durasi) {
        this.judul_1004 = judul;
        this.penyanyi_1004 = penyanyi;
        this.durasi_1004 = durasi;
    }

    @Override
    public String toString() {
        return judul_1004 + " - " + penyanyi_1004 + " (" + durasi_1004 +
" detik)";
    }
}
```

b. Kode program Sorting_2511531004

```
package Pekan8_2511531004;

import java.util.Scanner;

public class Sorting_2511531004 {
    Lagu_2511531004[] dataLagu_1004 = new Lagu_2511531004[20];
    int jumlah_1004 = 0;

    // Input minimal 7 lagu
    void inputData_1004() {
        dataLagu_1004[jumlah_1004++] = new Lagu_2511531004("La Perla",
"Juanes", 196);
        dataLagu_1004[jumlah_1004++] = new Lagu_2511531004("La Rumba Del
Perdon", "Carlos", 252);
        dataLagu_1004[jumlah_1004++] = new Lagu_2511531004("Mio Cristo
Piange Diamanti", "Andrea", 270);
        dataLagu_1004[jumlah_1004++] = new Lagu_2511531004("Shape of
You", "Ed Sheeran", 240);
        dataLagu_1004[jumlah_1004++] = new Lagu_2511531004("Fix You",
"Coldplay", 300);
        dataLagu_1004[jumlah_1004++] = new Lagu_2511531004("Yellow",
"Coldplay", 260);
    }
}
```

```

        dataLagu_1004[jumlah_1004++] = new Lagu_2511531004("Perfect",
"Ed Sheeran", 280);
    }

    // Tampil data
    void tampilData_1004() {
        for (int i = 0; i < jumlah_1004; i++) {
            System.out.println((i+1) + ". " + dataLagu_1004[i]);
        }
    }

    // === SHELL SORT (judul A-Z) ===
    void shellSort_1004() {
        int n = jumlah_1004;
        for (int gap = n/2; gap > 0; gap /= 2) {
            for (int i = gap; i < n; i++) {
                Lagu_2511531004 temp = dataLagu_1004[i];
                int j;
                for (j = i; j >= gap && dataLagu_1004[j-gap].judul_1004.compareTo(temp.judul_1004) > 0; j -= gap) {
                    dataLagu_1004[j] = dataLagu_1004[j-gap];
                }
                dataLagu_1004[j] = temp;
            }
        }
    }

    // === QUICK SORT (durasi ascending) ===
    void quickSort_1004(int low, int high) {
        if (low < high) {
            int pi = partition_1004(low, high);
            quickSort_1004(low, pi-1);
            quickSort_1004(pi+1, high);
        }
    }

    int partition_1004(int low, int high) {
        int pivot = dataLagu_1004[high].durasi_1004;
        int i = (low-1);
        for (int j = low; j < high; j++) {
            if (dataLagu_1004[j].durasi_1004 <= pivot) {
                i++;
                Lagu_2511531004 temp = dataLagu_1004[i];
                dataLagu_1004[i] = dataLagu_1004[j];
                dataLagu_1004[j] = temp;
            }
        }
        Lagu_2511531004 temp = dataLagu_1004[i+1];
        dataLagu_1004[i+1] = dataLagu_1004[high];
        dataLagu_1004[high] = temp;
        return i+1;
    }

    // === MERGE SORT (judul A-Z) ===
    void mergeSort_1004(int left, int right) {
        if (left < right) {
            int mid = (left+right)/2;

```

```

        mergeSort_1004(left, mid);
        mergeSort_1004(mid+1, right);
        merge_1004(left, mid, right);
    }
}

void merge_1004(int left, int mid, int right) {
    int n1 = mid-left+1;
    int n2 = right-mid;

    Lagu_2511531004[] L = new Lagu_2511531004[n1];
    Lagu_2511531004[] R = new Lagu_2511531004[n2];

    for (int i=0; i<n1; i++) L[i] = dataLagu_1004[left+i];
    for (int j=0; j<n2; j++) R[j] = dataLagu_1004[mid+1+j];

    int i=0, j=0, k=left;
    while (i<n1 && j<n2) {
        if (L[i].judul_1004.compareTo(R[j].judul_1004) <= 0) {
            dataLagu_1004[k++] = L[i++];
        } else {
            dataLagu_1004[k++] = R[j++];
        }
    }
    while (i<n1) dataLagu_1004[k++] = L[i++];
    while (j<n2) dataLagu_1004[k++] = R[j++];
}

// === DRIVER ===
public static void main(String[] args) {
    Sorting_2511531004 s = new Sorting_2511531004();
    s.inputData_1004();

    Scanner sc = new Scanner(System.in);
    System.out.println("=== Sorting Playlist NIM: 2511531004 ===");
    System.out.println("Pilih Algoritma (1=Shell, 2=Quick, 3=Merge):");

    int pilih = sc.nextInt();

    System.out.println("\nData Sebelum Sorting:");
    s.tampilData_1004();

    if (pilih == 1) {
        s.shellSort_1004();
        System.out.println("\nData Setelah Shell Sort (Judul A-Z):");
    } else if (pilih == 2) {
        s.quickSort_1004(0, s.jumlah_1004-1);
        System.out.println("\nData Setelah Quick Sort (Durasi Asc):");
    } else if (pilih == 3) {
        s.mergeSort_1004(0, s.jumlah_1004-1);
        System.out.println("\nData Setelah Merge Sort (Judul A-Z):");
    }
    s.tampilData_1004();
}

```

```
}
```

Output

```
=== Sorting Playlist NIM: 2511531004 ===
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge):
1

Data Sebelum Sorting:
1. La Perla - Juanes (196 detik)
2. La Rumba Del Perdon - Carlos (252 detik)
3. Mio Cristo Piange Diamanti - Andrea (270 detik)
4. Shape of You - Ed Sheeran (240 detik)
5. Fix You - Coldplay (300 detik)
6. Yellow - Coldplay (260 detik)
7. Perfect - Ed Sheeran (280 detik)

Data Setelah Shell Sort (Judul A-Z):
1. Fix You - Coldplay (300 detik)
2. La Perla - Juanes (196 detik)
3. La Rumba Del Perdon - Carlos (252 detik)
4. Mio Cristo Piange Diamanti - Andrea (270 detik)
5. Perfect - Ed Sheeran (280 detik)
6. Shape of You - Ed Sheeran (240 detik)
7. Yellow - Coldplay (260 detik)
```